

views::as_rvalue

Document #: P2446R2
Date: 2022-02-14
Project: Programming Language C++
Audience: LEWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>

Contents

- [1 Revision History](#)
- [2 Introduction](#)
 - [2.1 But why now?](#)
 - [2.2 Naming](#)
- [3 Wording](#)
 - [24.7.? As rvalue view \[range.as.rvalue\]](#)
 - [3.1 Feature-test macro](#)
- [4 References](#)

§ 1 Revision History

Since [[P2446R1](#)], renamed to `views::as_rvalue` and updated wording.

Since [[P2446R0](#)], renamed to `views::all_move` and added a feature-test macro.

§ 2 Introduction

In [[P2214R1](#)], I wrote:

Several of the above views that are labeled “not proposed” are variations on a common theme: `addressof`, `indirect`, and `move` are all basically wrappers around `transform` that take `std::addressof`, `std::dereference` (a function object we do not have at the moment), and `std::move`, respectively. Basically, but not exactly, since one of those functions doesn’t exist yet and the other three we can’t pass as an argument anyway.

But some sort of broader ability to pass functions into functions would mostly alleviate the need for these. `views::addressof` is shorter than `views::transform(LIFT(std::addressof))` (assuming a `LIFT` macro that wraps a name and emits a lambda), but we’re not sure that we necessarily need to add special cases of `transform` for every useful function.

While this is true for `views::addressof` and `views::indirect`, it's actually *not* correct for `views::move`. There is actually a lot more involvement here.

To start with, while if we had a range of lvalues, we would just want to `std::move()` each element of the range, that's not true if we had a range of rvalues. Those... we wouldn't really have to do anything with (indeed, if we had a range of prvalues, the extra `std::move()` would just add unnecessary overhead by materializing those objects earlier). Except in some cases, we *do* still want to do something with the prvalues - if we were `zip()`ing [P2321R2] one range of lvalues, we would get back a range of `tuple<T&>`. But the result of piping that into `views::move` shouldn't be `tuple<T&&&>` (as a naive `views::transform(std::move)` would do) and it shouldn't be `tuple<T&>` (as a slightly less naive implementation that avoids transforming rvalues) - we should get back a range of `tuple<T&&>`.

Indeed, we already have a customization point that does this for us: `ranges::iter_move` 23.3.3.1 [iterator.cust.move]. But we can't pass that into `views::transform`, because `ranges::iter_move` operators on the iterator (as the name suggest) and not the underlying element. We would need to spell this as `views::iter_transform(ranges::iter_move)`, but we don't yet have a `views::iter_transform` (although range-v3 does).

However, even if we could define `views::move` as simply `views::iter_transform(ranges::iter_move)`, that's still a poor definition, for a very important reason: `ranges::iter_move` (just like `std::move`) isn't just *some* arbitrary transformation. We know a lot about this particular one, and specifically we know that it consumes the source element. And as a result, that *should* limit the algorithms that you can perform on `views::move(e)` - such an adapted range needs to be an *input* range. `views::transform(e, std::move)` and `views::iter_transform(e, ranges::iter_move)` would both be potentially up to random-access, but we need to set a much much lower ceiling here.

As such, `views::move` needs to be a first-class range adaptor.

§ 2.1 But why now?

The above argues why `views::move` can't just be a `views::transform` or a `views::iter_transform`, but why do we need it now? Well, there are two answers to this.

First, with the imminent adoption of `ranges::to` [P1206R6], we spent a lot of time on that paper trying to make collecting of elements as efficient as possible. As efficient as possible certainly includes *moving* elements instead of *copying* elements, where appropriate. However, right now, it's really up to users to figure out how to do that. A `views::move` would go a long way in making this as effortless as possible (and mirrors how users would move a single element).

Second, unlike `views::as_const` [P2278R1], where a lot of design effort had to be spent figuring out how to implement a constant iterator, that work has already been done for move iterators.

`std::move_iterator<I>` 23.5.3 [move.iterators] exists and already does the right thing, and there's already a `std::move_sentinel<S>` 23.5.3.10 [move.sentinel] to handle non-common ranges. All the work is done, we just have to put it together. As you can see below, the wording is pretty small and very straightforward.

This is a tiny paper, and while this should be considered the lowest priority of all the range adaptors (as the C++23 Ranges Plan doesn't even propose it at all), we may as well at least try to squeeze it in.

§ 2.2 Naming

In range-v3, the adaptors to move all of the elements and make all of the elements const were named `views::move` and `views::const_`, respectively. [P2446R0] and [P2278R0] simply used range-v3's names. [P2278R1] changed from `views::const_` to `views::as_const` (as an analogue for `std::as_const`). But during LEWG telecon discussion of these papers [p2278-minutes] [p2446-minutes], it was suggested that having such names is confusing because of the ambiguity between whether these adaptors operate on the *range* or the *elements* thereof, and so we end up with the names `views::all_move` and `views::all_const` to avoid conflict with the two `std::move`s we already have and the `std::as_const`, that do something else.

I think these names are pretty bad (it's not named `views::all_transform?`), and there was a follow-up paper to suggest reverting this rename [P2501R0]. Ensuing discussion pointed out a potential issue that because `std::move` and `std::views::move` can both be used unary and unqualified, there's potential for code silently changing meaning if `move` were previously used unqualified. Example from Nicolai Josuttis:

```
using namespace std::views;

std::vector<std::string> v1{"hello", "world"};
auto v2 = move(v1);           // OOPS: initializes a view to v1
```

If the user had done a `using namespace std::views` (to do some ranges pipeline work without having to qualify `std::views::meow`) but then had previously used `move(x)` on some object with `std` as an associated namespace, in C++20 this would invoke `std::move` whereas in C++23, if we add this range adaptor under the name `std::views::move`, this would invoke the adaptor instead. The moral of the story here is probably to just always qualify `std::move` (indeed, the library specification always uses it qualified), but there was sufficient concern about this example that there was no consensus in LEWG to revert and go back to `views::move`, preferring `views::all_move`.

Nevertheless, `all_move` is just not a great name, and it's also the wrong tense. A subsequent suggestion from Ville Voutilainen is `views::as_rvalue`. This has the advantage of being the correct tense and not conflicting with anything else in the standard, and the result of this adaptor is that you do end up with a range of rvalues (whether the adaptor actually does something or not. Indeed that's what the first sentence of the introductory wording has always said). An alternative name I would entertain would be `views::move_each` (but not `views::move_elements`, since `views::elements<N>` exists and these don't quite line up).

§ 3 Wording

Add to 24.2 [ranges.syn]:

```
#include <compare>           // see [compare.syn]
#include <initializer_list>   // see [initializer.list.syn]
#include <iterator>           // see [iterator.synopsis]
```

```

namespace std::ranges {
    // ...
+
+ template<view V>
+     requires input_range<V>
+     class as_rvalue_view;
+
+ template<class T>
+     inline constexpr bool enable_borrowed_range<as_rvalue_view<T>> =
+         enable_borrowed_range<T>;
+
+ namespace views { inline constexpr unspecified as_rvalue = unspecified; }

    // ...
}

```

§ 24.7.? As rvalue view [range.as.rvalue]

§ 24.7.?.1 Overview [range.as.rvalue.overview]

- 1 as_rvalue_view presents a view of an underlying sequence with the same behavior as the underlying sequence except that its elements are rvalues. Some generic algorithms can be called with a as_rvalue_view to replace copying with moving.
- 2 The name `views::as_rvalue` denotes a range adaptor object ([range.adaptor.object]). Let *E* be an expression and let *T* be `decltype((E))`. The expression `views::as_rvalue(E)` is expression-equivalent to:
 - (2.1) — `views::all(E)` if `same_as<range_rvalue_reference_t<T>, range_reference_t<T>>` is `true`
 - (2.2) — Otherwise, `ranges::as_rvalue_view(E)`.

3 [Example:

```

vector<string> words = {"the", "quick", "brown", "fox", "ate", "a",
    "pterodactyl"};
vector<string> new_words;
ranges::copy(words | views::as_rvalue, back_inserter(new_words)); // moves
    each string from words into new_words

```

-end example]

§ 24.7.?.2 Class template as_rvalue_view [range.as.rvalue.view]

```

namespace std::ranges {
    template<input_range V>
        requires view<V>
        class as_rvalue_view : public view_interface<as_rvalue_view<V>>
        {
            V base_ = V(); // exposition only

        public:

```

```

as_rvalue_view() requires default_initializable<V> = default;
constexpr explicit as_rvalue_view(V base);

constexpr V base() const& requires copy_constructible<V> { return base_;
}
constexpr V base() && { return std::move(base_); }

constexpr auto begin() requires (!simple-view<V>) { return
    move_iterator(ranges::begin(base_)); }
constexpr auto begin() const requires range<const V> { return
    move_iterator(ranges::begin(base_)); }

constexpr auto end() requires (!simple-view<V>) {
    if constexpr (common_range<V>) {
        return move_iterator(ranges::end(base_));
    } else {
        return move_sentinel(ranges::end(base_));
    }
}
constexpr auto end() const requires range<const V> {
    if constexpr (common_range<const V>) {
        return move_iterator(ranges::end(base_));
    } else {
        return move_sentinel(ranges::end(base_));
    }
}

constexpr auto size() requires sized_range<V> { return
    ranges::size(base_); }
constexpr auto size() const requires sized_range<const V> { return
    ranges::size(base_); }
};

template<class R>
as_rvalue_view(R&&) -> as_rvalue_view<views::all_t<R>>;
}

constexpr explicit as_rvalue_view(V base);

```

¹ *Effects*: Initializes *base_* with `std::move(base)`.

§ 3.1 Feature-test macro

Add the following macro definition to 17.3.2 [\[version.syn\]](#), with the value selected by the editor to reflect the date of adoption of this paper:

```
#define __cpp_lib_ranges_as_rvalue 20XXXXL // also in <ranges>
```

§ 4 References

[P1206R6] Corentin Jabot, Eric Niebler, Casey Carter. 2021-08-03. Conversions from ranges to containers.

<https://wg21.link/p1206r6>

[P2214R1] Barry Revzin, Conor Hoekstra, Tim Song. 2021-09-14. A Plan for C++23 Ranges.

<https://wg21.link/p2214r1>

[p2278-minutes] LEWG. 2021. LEWG minutes for P2278.

<https://wiki.edg.com/bin/view/Wg21telecons2021/P2278#Library-Evolution-2021-11-09>

[P2278R0] Barry Revzin. 2021-01-10. cbegin should always return a constant iterator.

<https://wg21.link/p2278r0>

[P2278R1] Barry Revzin. 2021-09-15. cbegin should always return a constant iterator.

<https://wg21.link/p2278r1>

[P2321R2] Tim Song. 2021-06-11. zip.

<https://wg21.link/p2321r2>

[p2446-minutes] LEWG. 2021. LEWG minutes for P2446.

<https://wiki.edg.com/bin/view/Wg21telecons2021/P2446#Library-Evolution-2021-11-09>

[P2446R0] Barry Revzin. 2021-09-18. views::move.

<https://wg21.link/p2446r0>

[P2446R1] Barry Revzin. 2021-11-17. views::all_move.

<https://wg21.link/p2446r1>

[P2501R0] Ville Voutilainen. 2021-12-14. Undo the rename of views::move and views::as_const.

<https://wg21.link/p2501r0>